

● CLOUD COST · SETUP GUIDE

Sending AWS China Spend to CloudZero in USD

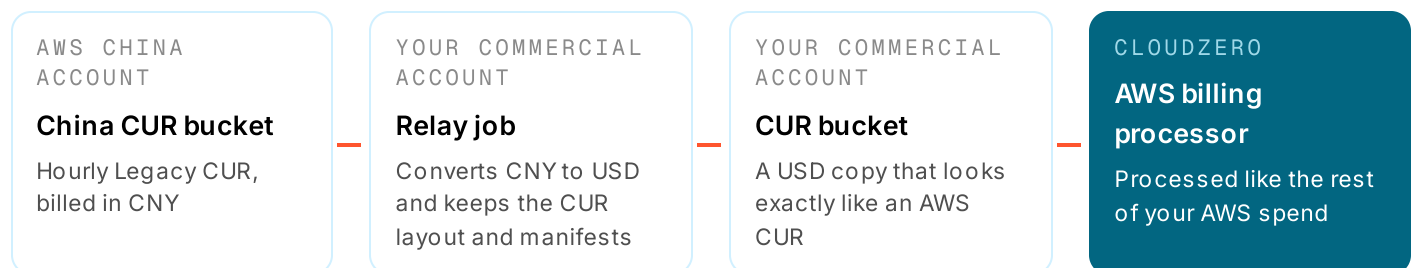
For teams bringing AWS China (Beijing and Ningxia) billing data into CloudZero, converted to US dollars.

REGIONS	YOU RUN	CLOUDZERO DOES
Beijing, Ningxia	A scheduled relay job	A one-time connection switch

This guide walks your team through bringing AWS China spend into CloudZero, converted to US dollars. AWS China runs in its own partition, so CloudZero can't read it through your standard AWS connection. Instead, a small relay job that you run copies your China Cost and Usage Report (CUR) into a bucket in your commercial AWS account on a schedule. It converts yuan to dollars along the way and keeps the exact CUR layout.

Because the output is a valid AWS CUR, your CloudZero contact switches the connection to CloudZero's AWS billing processor. China spend then shows up with the same services, usage types and cost types as the rest of your AWS spend.

How It Works



The relay reads each new version of the China CUR, multiplies every money column by your finance team's monthly exchange rate, sets the currency columns to USD, and writes the result to your commercial bucket. Folder names, file names and manifests match what AWS produces, so CloudZero reads it like any other CUR.

● YOU OWN THE CONVERSION

You run the relay job and control the exchange rates. CloudZero makes a one-time change to the connection and otherwise processes the data as delivered.

Before You Start

● CLEAR DATA TRANSFER FIRST

Get sign-off from your legal or compliance team before you build anything.

This setup copies billing data out of mainland China. That data includes account IDs, resource IDs and tags, so confirm the transfer is acceptable under your cross-border data transfer policies.

Prerequisites

- An AWS China account in Beijing (`cn-north-1`) or Ningxia (`cn-northwest-1`), with access to the Billing console and IAM
- A commercial AWS account that's already connected to CloudZero. The destination bucket and relay job live here.
- Permission to create connections in CloudZero (**Settings** → **Cloud Connections**)
- An agreed CNY to USD rate policy from finance, typically the monthly rate they already report with
- Someone who can deploy a Python Lambda, or a container task for large CURs, in the commercial account

● STEP 01

Confirm the China Cost and Usage Report

In the China account's Billing console, check for an existing Legacy CUR export and compare it with the settings below.

- **If one exists and matches**, reuse it. It already holds the history you'll backfill.
- **If one exists but doesn't match**, change its settings to match, where the console allows it. Before you do, check that nothing else depends on its current settings.
- **If none exists**, or a setting can't be changed on the existing report, create a new Legacy CUR export with these settings:

SETTING	REQUIRED VALUE
Report type	Legacy CUR export
Time granularity	Hourly
Report versioning	Create new report version
Compression and format	GZIP, CSV
Include resource IDs	On
Data refresh	Automatic
Split cost allocation data	Off

Note the **bucket name**, **report path prefix**, **report name** and bucket **region**. You'll need all four in Step 5.

● CHECK THE CURRENCY

Open one report file and look at `lineItem/CurrencyCode`. It's normally `CNY`. If it's already `USD`, the relay copies values unchanged and the rest of this guide still applies.

● STEP 02

Grant Read Access to the China CUR Bucket

IAM roles can't be assumed across partitions, so the relay needs an access key. In the China account, create an IAM user named `cloudzero-cur-reader` with no console access, and attach this policy. Replace the bucket and prefix with your values, and keep the `arn:aws-cn` partition.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws-cn:s3:::YOUR_CHINA_CUR_BUCKET",
      "Condition": { "StringLike": { "s3:prefix": [ "YOUR_CUR_PREFIX/*" ] } }
    },
    {
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws-cn:s3:::YOUR_CHINA_CUR_BUCKET / YOUR_CUR_PREFIX /*"
    }
  ]
}
```

REPLACE BEFORE PASTING

- `YOUR_CHINA_CUR_BUCKET`: the China account's CUR bucket
- `YOUR_CUR_PREFIX`: the China report's path prefix (Step 1)

Create an access key for the user. In the **commercial** account, store it in Secrets Manager as a secret named `aws-china-cur-reader`:

```
{ "access_key_id": "YOUR_ACCESS_KEY_ID", "secret_access_key": "YOUR_SECRET_ACCESS_KEY" }
```

REPLACE BEFORE PASTING

- `YOUR_ACCESS_KEY_ID`: the access key ID for the China IAM user
- `YOUR_SECRET_ACCESS_KEY`: the secret access key for the China IAM user

● ROTATE THIS KEY

It's a long-lived key, so rotate it on your normal schedule (every 90 days is common). Its only permission is reading the CUR prefix.

● STEP 03

Create the Destination Bucket

In the commercial account that's connected to CloudZero, open the S3 console and select **Create bucket**. Use these settings:

SETTING	VALUE
Bucket type	General purpose
Bucket name	Any available name. S3 bucket names are unique across all AWS accounts, so adding your account ID helps. Later steps call this <code>YOUR_DEST_BUCKET</code> .
AWS Region	The Region you'll deploy the relay Lambda in, so the copy stays in one Region
Object Ownership	ACLs disabled (Bucket owner enforced)
Block Public Access	Block all public access: on
Bucket Versioning	Disable. Each relay run writes a new report version to its own folder.
Default encryption	SSE-S3 (Amazon S3 managed keys). SSE-KMS also works, but then the relay role and the CloudZero role both need <code>kms:Decrypt</code> and <code>kms:GenerateDataKey</code> on the key.
Bucket policy	None. Access comes from IAM policies: the relay role in Step 5 and the CloudZero role in Step 7.

You don't need to create any folders. The relay creates `china-cur/` and `cur-relay-audit/` on its first run, and you'll add `fx/` in Step 4. Once it's running, the bucket looks like this:

```
YOUR_DEST_BUCKET/  
  china-cur/                                # report path prefix  
    YOUR_REPORT_NAME/                      # report name, same as in China  
      20260901-20261001/                   # billing period, as AWS names it  
        YOUR_REPORT_NAME-Manifest.json     # top-level manifest: current version  
        f8cc6fc9-.../                     # assembly (report version) folder  
          YOUR_REPORT_NAME-00001.csv.gz  
          YOUR_REPORT_NAME-Manifest.json  
  fx/  
    cny_usd_monthly.json                   # rates from finance (Step 4)  
    cur-relay-audit/                      # CNY and USD totals per version
```

REPLACE BEFORE PASTING

- `YOUR_DEST_BUCKET`: the destination bucket you created in Step 3
- `YOUR_REPORT_NAME`: the China report's name (Step 1)

● DON'T EXPIRE THE REPORT FOLDER BY AGE

A closed month's current version stops changing, so an age-based lifecycle rule would eventually delete it. Remove superseded assembly folders by hand after a month is closed, or leave them.

● STEP 04

Publish the Monthly Exchange Rates

The relay converts with one rate per month, read from a JSON file that finance owns. Each value is **USD per 1 CNY**.

```
# s3://YOUR_DEST_BUCKET/fx/cny_usd_monthly.json (illustrative rates)
{
  "2026-07": "0.1395",
  "2026-08": "0.1401",
  "2026-09": "0.1398"
}
```

REPLACE BEFORE PASTING

- `YOUR_DEST_BUCKET`: the destination bucket you created in Step 3
- The months and rates: use the ones your finance team publishes (these are examples)

Upload it to the destination bucket under the `fx/` prefix, either with **Create folder** in the S3 console or from the CLI:

```
aws s3 cp cny_usd_monthly.json s3://YOUR_DEST_BUCKET/fx/cny_usd_monthly.json
```

REPLACE BEFORE PASTING

- `YOUR_DEST_BUCKET`: the destination bucket you created in Step 3

The bucket must match `DEST_BUCKET` and the key must match `FX_RATES_KEY` (Step 5). Add the current month's working rate at the start of each month, and add a rate for every historical month you plan to backfill.

● NO RATE, NO RUN

If the month being processed has no rate, the relay stops instead of guessing. Yuan can never land in CloudZero labeled as dollars.

● STEP 05

Deploy the Relay Job

The relay is one Python Lambda function. Each run rebuilds the current month from the China report, plus the previous month for the first 10 days while AWS finalizes it. It skips any month where neither the China report nor the rate has changed, so frequent runs cost nothing.

1. Create the execution role

1. In the IAM console, go to **Roles** and select **Create role**.
2. Set **Trusted entity type** to **AWS service** and **Use case** to **Lambda**, then select **Next**.
3. Search for and select `AWSLambdaBasicExecutionRole`, which lets the function write logs. Select **Next**.
4. Name the role `cloudzero-china-cur-relay` and select **Create role**.
5. Open the new role, then select **Add permissions → Create inline policy**.
6. Switch to **JSON**, paste the policy below, select **Next**, name it `relay-access` and select **Create policy**.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "secretsmanager:GetSecretValue",
      "Resource": "arn:aws:secretsmanager:YOUR_REGION:YOUR_ACCOUNT_ID:secret:aws-china-cur-reader-*"
    },
    {
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::YOUR_DEST_BUCKET"
    },
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": [
        "arn:aws:s3:::YOUR_DEST_BUCKET/china-cur/*",
        "arn:aws:s3:::YOUR_DEST_BUCKET/cur-relay-audit/*",
        "arn:aws:s3:::YOUR_DEST_BUCKET/fx/*"
      ]
    }
  ]
}
```

REPLACE BEFORE PASTING

- **YOUR_REGION**: the commercial Region you deployed the relay in
- **YOUR_ACCOUNT_ID**: your commercial account ID
- **YOUR_DEST_BUCKET**: the destination bucket you created in Step 3

2. Create the function

1. In the Lambda console, select **Create function → Author from scratch**.
2. Name it **cloudzero-china-cur-relay** and set **Runtime** to **Python 3.12**.
3. Open **Additional settings** and turn on **Custom execution role**. In the panel that opens, choose **cloudzero-china-cur-relay** and select **Save**. Then select **Create function**.
4. On the **Code** tab, replace everything in **lambda_function.py** with the code in [Appendix A](#), then select **Deploy**. The runtime already includes boto3, so there's nothing to install.

3. Configure it

On the **Configuration** tab, open **General configuration → Edit** and set:

Memory	2048 MB
Ephemeral storage	10240 MB. Report files are staged on disk before upload.
Timeout	15 min 0 sec. If a month's CUR needs longer, run the same code as an ECS Fargate or AWS Batch task.

Then open **Environment variables** → **Edit** and add each of these:

VARIABLE	EXAMPLE	PURPOSE
CN_REGION	cn-northwest-1 or cn-north-1	Region of the China CUR bucket, from Step 1
CN_CUR_BUCKET	YOUR_CHINA_CUR_BUCKET	China CUR bucket
CN_CUR_PREFIX	YOUR_CUR_PREFIX	China report path prefix
CUR_REPORT	YOUR_REPORT_NAME	Report name, kept the same in the destination
CN_SECRET_ID	aws-china-cur-reader	Secret from Step 2
DEST_BUCKET	YOUR_DEST_BUCKET	Destination bucket from Step 3
DEST_PREFIX	china-cur	Destination report path prefix
FX_RATES_KEY	fx/cny_usd_monthly.json	Rates file from Step 4

4. Schedule it

1. At the top of the function page, select **Add trigger** and choose **EventBridge (CloudWatch Events)**.
2. Select **Create a new rule**, name it `cloudzero-china-cur-relay-schedule`, choose **Schedule expression** and enter `cron(0 */8 * * ? *)`. That runs every 8 hours, and AWS refreshes the CUR up to three times a day.
3. Select **Add**.

● NO RESOURCE-BASED POLICY NEEDED

Skip **Configuration** → **Permissions** → **Resource-based policy statements**, the form that asks for an AWS account or service and a statement ID. That controls who can invoke the function, and adding the EventBridge trigger creates that statement for you.

5. Test it

On the **Test** tab, create an event with `{}` as the body and select **Test**.

● EXPECTED RESULT

The log shows one line per month, such as `20260901-20261001: 184220 rows, CNY 1203344.18 -> USD 168227.52 at 0.1398`, and an audit record appears under `cur-relay-audit/`. Its `converted_columns` list shows exactly which columns were converted for your report. Review it before you rely on the schedule.

● STEP 06

Backfill History

On the function's **Test** tab, run a test event with the historical months you want in CloudZero as the body:

```
{"periods": ["2026-06", "2026-07", "2026-08"]}
```

Each month needs a rate in the rates file first. Backfilling before Step 7 means CloudZero's first pass includes your history.

● STEP 07

Connect the Bucket and Request AWS Processing

Create a connection so CloudZero can read the bucket:

1. In CloudZero, go to **Settings → Cloud Connections** and select **Create Connection +**.
2. Select **Custom**, then the **S3 Bucket** tile.
3. Set **Connection Name** to `aws-china-cur` and **Cloud Provider** to `AWS China`.
4. For **AWS Connection**, choose the commercial account that holds the bucket.
5. Set **Bucket Name** to `YOUR_DEST_BUCKET` and **Bucket Path** to `china-cur/YOUR_REPORT_NAME`, with no slashes at either end.
6. Select **Generate IAM Policy** and attach the policy, as an inline policy, to the IAM role CloudZero uses for that account. For accounts connected with CloudZero's automated setup it's named like `cloudzero-connected-account-live-ResourceOwner-Role-<ID>`. For manually connected accounts it's the role you created, shown as the **Role ARN** on that account's connection in CloudZero.
7. Select **Save**.

Then send your CloudZero contact the connection name, bucket name, bucket path and report name, and ask them to switch the connection to AWS billing processing.

● AN ERROR UNTIL THE SWITCH IS EXPECTED

Until your CloudZero contact makes the switch, the connection shows **Pending Data** or an error, because CloudZero is still reading the path as a custom cost source. After the switch, CloudZero labels this spend by China region (Beijing or Ningxia), based on the CUR's billing entity.

● STEP 08

Validate the Numbers

Allow up to 24 hours after the switch for the first processing run. Then check:

1. In the latest audit record, `unblended_usd` equals `unblended_cny` times the rate.
2. In CloudZero Explorer, select the month, filter to the China accounts and use **Billed Cost**. The total matches the audit record's `unblended_usd`.
3. `unblended_cny` matches the China invoice for the month. Small differences are normal until AWS finalizes the invoice.

● EXPECTED RESULT

China accounts appear in Explorer in USD, with services and usage types broken out the same way as your global AWS accounts.

● STEP 09

Close Each Month

When finance publishes the final monthly rate and AWS finalizes the China invoice (usually in the first few days of the next month), update the rate in `cny_usd_monthly.json`. Then run a test event for that month on the function's **Test** tab:

```
{"periods": ["2026-09"]}
```

The rate change produces a new report version, and CloudZero replaces the month with it. Add this to your month-end close checklist, next to setting the new month's working rate.

● REFERENCE

Which Columns Get Converted

The relay converts a column when it's in a money category (`lineItem`, `pricing`, `reservation`, `savingsPlan` or `discount`) and its name ends in *Cost*, *Rate*, *Fee*, *Commitment*, *Discount* or *UpfrontValue*. Blank values stay blank.

CATEGORY	TYPICAL CONVERTED COLUMNS
<code>lineItem</code>	UnblendedCost, UnblendedRate, BlendedCost, BlendedRate, NetUnblendedCost, NetUnblendedRate
<code>pricing</code>	publicOnDemandCost, publicOnDemandRate
<code>reservation</code>	EffectiveCost, AmortizedUpfrontCostForUsage, AmortizedUpfrontFeeForBillingPeriod, RecurringFeeForUsage, UnusedAmortizedUpfrontFeeForBillingPeriod, UnusedRecurringFee, UpfrontValue, and their Net variants
<code>savingsPlan</code>	SavingsPlanEffectiveCost, SavingsPlanRate, TotalCommitmentToDate, UsedCommitment, AmortizedUpfrontCommitmentForBillingPeriod, RecurringCommitmentForBillingPeriod, and their Net variants
<code>discount</code>	TotalDiscount, EdpDiscount, BundledDiscount, PrivateRateDiscount

`lineItem/CurrencyCode` and `pricing/currency` are set to `USD`. Usage amounts, rate codes, IDs, counts and tags are never changed.

Troubleshooting

1. **Connection stays on Pending Data.** The switch to AWS billing processing hasn't been made yet. Follow up with your CloudZero contact.
2. **China costs look about 7x too high.** Yuan landed as dollars. Check that nothing other than the relay writes to the report folder, and that the audit record lists the expected converted columns.
3. **"No CNY to USD rate for YYYY-MM".** Add that month to the rates file and re-run.
4. **"Unexpected currency" error.** The report contains a currency other than CNY or USD. Stop and contact your CloudZero contact before continuing.
5. **AccessDenied reading from China.** Check that `CN_REGION` is a `cn-` region, the policy uses `arn:aws-cn:`, and the secret holds the current key.
6. **AccessDenied on the destination.** The execution role needs `s3:ListBucket` on the destination bucket to check for existing manifests.
7. **A month didn't update after a rate change.** The relay only rebuilds months in its current window. Run a test event with that month in `periods`.
8. **Lambda times out or runs out of disk.** The CUR is too large for Lambda. Run the same code as a Fargate or Batch task.
9. **A tag is missing.** Activate it as a cost allocation tag in the China Billing console. It appears in the CUR from then on.

Relay Code

The complete relay job from Step 5. Paste it into `lambda_function.py`, replacing the starter code. The default handler, `lambda_function.lambda_handler`, matches the code, so leave it as is.

```
"""Relay the AWS China CUR into a commercial-partition bucket, converted from CNY
to USD, keeping the Legacy CUR layout and manifests so CloudZero can process it
as native AWS billing data."""
import csv
import gzip
import io
import json
import os
import re
import re
import tempfile
import uuid
from datetime import date, datetime, timedelta, timezone
from decimal import Decimal, InvalidOperation

import boto3

CN_REGION = os.environ["CN_REGION"].strip()          # cn-north-1 or cn-northwest-1
CN_CUR_BUCKET = os.environ["CN_CUR_BUCKET"].strip()
CN_CUR_PREFIX = os.environ["CN_CUR_PREFIX"].strip()  # report path prefix in China
CUR_REPORT = os.environ["CUR_REPORT"].strip()        # report name, kept the same
CN_SECRET_ID = os.environ["CN_SECRET_ID"].strip()
DEST_BUCKET = os.environ["DEST_BUCKET"].strip()
DEST_PREFIX = os.environ["DEST_PREFIX"].strip()      # report path prefix in the destination
FX_RATES_KEY = os.environ["FX_RATES_KEY"].strip()
AUDIT_PREFIX = os.environ.get("AUDIT_PREFIX", "cur-relay-audit").strip()
LOOKBACK_DAYS = int(os.environ.get("LOOKBACK_DAYS", "10").strip())

# A column is converted when its category holds money and its name ends in one
# of these words, e.g. lineItem/UnblendedCost, pricing/publicOnDemandRate,
# savingsPlan/TotalCommitmentToDate, reservation/UpfrontValue.
MONEY_CATEGORIES = {"lineItem", "pricing", "reservation", "savingsPlan", "discount"}
MONEY_SUFFIX = re.compile(r"(Cost|Rate|Fee|Commitment|Discount|UpfrontValue)$")
CURRENCY_COLUMNS = {"lineItem/CurrencyCode", "pricing/currency"}
ONE = Decimal(1)

def dec(value):
    try:
        return Decimal(value) if value else Decimal(0)
    except InvalidOperation:
        return Decimal(0)

PLACES = Decimal("0.0000000001") # CUR money columns carry at most 10 decimal places

def scale(value, factor):
    try:
        return format((Decimal(value) * factor).quantize(PLACES).normalize(), "f")
    except InvalidOperation:
        return value # leave anything non-numeric untouched

def is_money(column):
    category, _, name = column.partition("/")
    return category in MONEY_CATEGORIES and bool(MONEY_SUFFIX.search(name))
```

```

def next_month(d):
    return (d.replace(day=28) + timedelta(days=4)).replace(day=1)

def billing_period_id(start):
    return f"{start:%Y%m%d}-{next_month(start):%Y%m%d}"

def periods_to_process(event):
    if event and event.get("periods"): # {"periods": ["2026-08"]} restates a month
        return [date.fromisoformat(p + "-01") for p in event["periods"]]
    today = datetime.now(timezone.utc).date()
    current = today.replace(day=1)
    periods = [current]
    if today.day <= LOOKBACK_DAYS: # AWS finalizes last month early in this one
        periods.insert(0, (current - timedelta(days=1)).replace(day=1))
    return periods

def china_s3():
    secret = json.loads(
        boto3.client("secretsmanager").get_secret_value(SecretId=CN_SECRET_ID)["SecretString"]
    )
    # A cn-* region makes boto3 use the amazonaws.com.cn endpoints
    return boto3.client(
        "s3",
        region_name=CN_REGION,
        aws_access_key_id=secret["access_key_id"],
        aws_secret_access_key=secret["secret_access_key"],
    )

def read_json(s3, bucket, key):
    try:
        return json.loads(s3.get_object(Bucket=bucket, Key=key)["Body"].read())
    except s3.exceptions.NoSuchKey:
        return None

def relay_file(cn, dest, src_key, dst_key, rate, totals):
    """Stream one CUR file, convert money columns to USD, upload it to the destination."""
    fd, path = tempfile.mkstemp(suffix=".csv.gz")
    os.close(fd)
    body = cn.get_object(Bucket=CN_CUR_BUCKET, Key=src_key)["Body"]
    with gzip.GzipFile(fileobj=body) as gin, \
        gzip.open(path, "wt", encoding="utf-8", newline="") as out:
        reader = csv.reader(io.TextIOWrapper(gin, encoding="utf-8", newline=""))
        writer = csv.writer(out, lineterminator="\n")
        header = next(reader)
        writer.writerow(header)
        money = [i for i, col in enumerate(header) if is_money(col)]
        currency = [i for i, col in enumerate(header) if col in CURRENCY_COLUMNS]
        code_i = header.index("lineItem/CurrencyCode")
        cost_i = header.index("lineItem/UnblendedCost")
        for row in reader:
            code = (row[code_i] or "CNY").upper()
            if code == "CNY":
                factor = rate
                totals["unblended_cny"] += dec(row[cost_i])
            elif code == "USD":
                factor = ONE

```

```

        else:
            raise ValueError(f"Unexpected currency {code} in {src_key}")
        for i in money:
            if row[i]:
                row[i] = scale(row[i], factor)
        for i in currency:
            if row[i]:
                row[i] = "USD"
        totals["unblended_usd"] += dec(row[cost_i])
        totals["rows"] += 1
        writer.writerow(row)
    dest.upload_file(path, DEST_BUCKET, dst_key)
    os.remove(path)
    return [header[i] for i in money]

def process_period(cn, dest, start, rates):
    period = billing_period_id(start)
    month = f"{start:%Y-%m}"
    if month not in rates:
        raise RuntimeError(f"No CNY to USD rate for {month} in {FX_RATES_KEY}")
    rate = Decimal(str(rates[month]))

    manifest_name = f"{CUR_REPORT}-Manifest.json"
    src_dir = f"{CN_CUR_PREFIX}/{CUR_REPORT}/{period}"
    dst_dir = f"{DEST_PREFIX}/{CUR_REPORT}/{period}"

    source = read_json(cn, CN_CUR_BUCKET, f"{src_dir}/{manifest_name}")
    if source is None:
        print(f"{period}: no China CUR yet, skipping")
        return

    # A new assembly ID whenever AWS publishes a new report version or the rate
    # changes, so CloudZero sees a new version to process. Otherwise, skip.
    assembly = str(uuid.uuid5(uuid.NAMESPACE_URL, f"{source['assemblyId']}|{rate}"))
    current = read_json(dest, DEST_BUCKET, f"{dst_dir}/{manifest_name}")
    if current and current.get("assemblyId") == assembly:
        print(f"{period}: already up to date")
        return

    totals = {"rows": 0, "unblended_cny": Decimal(0), "unblended_usd": Decimal(0)}
    dest_keys, converted = [], set()
    for src_key in source["reportKeys"]:
        dst_key = f"{dst_dir}/{assembly}/{src_key.rsplit('/', 1)[-1]}"
        converted.update(relay_file(cn, dest, src_key, dst_key, rate, totals))
        dest_keys.append(dst_key)

    # Same manifest AWS wrote, pointed at the new bucket, assembly and files
    manifest = {k: v for k, v in source.items() if k != "additionalArtifactKeys"}
    manifest.update({"assemblyId": assembly, "bucket": DEST_BUCKET, "reportKeys": dest_keys})
    body = json.dumps(manifest, indent=2)
    dest.put_object(Bucket=DEST_BUCKET, Key=f"{dst_dir}/{assembly}/{manifest_name}",
                    Body=body, ContentType="application/json")

    dest.put_object(
        Bucket=DEST_BUCKET,
        Key=f"{AUDIT_PREFIX}/{period}/{assembly}.json",
        Body=json.dumps({
            "period": period,
            "source_assembly_id": source["assemblyId"],
            "assembly_id": assembly,
            "rate_usd_per_cny": str(rate),
            "rows": totals["rows"],

```

```

        "unblended_cny": format(totals["unblended_cny"].quantize(Decimal("0.01")), "f"),
        "unblended_usd": format(totals["unblended_usd"].quantize(Decimal("0.01")), "f"),
        "converted_columns": sorted(converted),
    }, indent=2,
    ContentType="application/json",
)

# Written last: the top-level manifest is what points readers at the new version
dest.put_object(Bucket=DEST_BUCKET, Key=f"{dst_dir}/{manifest_name}",
                Body=body, ContentType="application/json")
print(f"{period}: {totals['rows']} rows, CNY {totals['unblended_cny']} "
      f"-> USD {totals['unblended_usd']} at {rate}")

def lambda_handler(event=None, context=None):
    cn = china_s3()
    dest = boto3.client("s3")
    rates = read_json(dest, DEST_BUCKET, FX_RATES_KEY)
    if rates is None:
        raise RuntimeError(f"Rates file {FX_RATES_KEY} not found in {DEST_BUCKET}")
    for start in periods_to_process(event):
        process_period(cn, dest, start, rates)

handler = lambda_handler # alias, if you save the file as relay.py and use relay.handler

```

● SUPPORT

Need a hand?

If you hit issues, reach out to your CloudZero contact. Include:

- The connection name and bucket path
- The billing period affected
- The audit record for that period, from `cur-relay-audit/`
- The relay's log output from the failing run



© CloudZero · Sending AWS China Spend to CloudZero in USD · Last updated 2026-09-24